PromptABI: Static Verification for the Discrete Interface Layer of LLM Systems

PromptABI Contributors

June 4, 2026

Abstract

Large language model applications increasingly fail at a layer that is neither neural nor traditionally tested: the discrete interface layer made of chat templates, tokenizer metadata, special tokens, stop policies, structured output schemas, tool definitions, provider request envelopes, and framework truncation rules. PromptABI is a static and differential verifier for that layer. Its core claim is deliberately narrow and therefore practical: before loading model weights or running inference, a developer can prove that important prompt and protocol states are possible, impossible, ambiguous, or unsafe under the exact artifacts that will be used in production.

This paper presents the repository as it now stands and the research system it is designed to become. The implementation already contains a typed Python package, a deterministic command line entrypoint, a public embedding API, diagnostic data structures, a minimal verification session, focused tests, examples, a fixture corpus layout, a CPU-only benchmark, documentation, and contribution guidance. The remaining checkers fit into that surface rather than requiring a rewrite. We describe the transducer-oriented formalism, the artifact model, the diagnostic contract, the first benchmark line, and the planned evaluation against real tokenizer, template, structured-output, provider, and budget failures. The result is a path toward a tool that is useful in CI and credible as a systems paper: it catches failures that arise before any model logit is relevant.

1 Motivation

Modern LLM applications are built from small, fragile protocol objects. A chat template decides which bytes separate roles. A tokenizer decides whether those bytes are ordinary content or special tokens. A stop policy decides where output is truncated. A tool schema decides which JSON fragment is valid. A provider adapter decides whether a tool call appears as a message, a delta, a function object, or a model-specific sentinel. A framework budget policy decides which messages, retrieved chunks, and tool definitions survive when the prompt exceeds the context window.

These objects compose into a pipeline:

```
messages -> chat template -> byte/string prompt -> tokenizer -> token stream
        -> constrained decoder / stop logic / tool parser -> application parser
```

The failure modes are concrete. A user field can contain a string that renders as an assistant delimiter. A stop string can fire inside a valid JSON string, making the application parse a malformed prefix. A tokenizer update can change special-token IDs while tests still pass because they never inspect the token stream. A RAG prompt can preserve the latest user message while silently dropping the safety instruction or the tool definition that gives the message meaning. These are not model-behavior questions. They are contract questions.

PromptABI treats those contracts as first-class verification targets. It does not promise semantic prompt-injection resistance. It does not ask whether a model will follow an instruction. It asks whether the interface representation admits a structural state that the developer did not intend. That boundary makes the tool useful without GPUs, private model access, or nondeterministic generation.

2 Current Repository Artifact

The current repository milestone establishes the skeleton needed for serious verification work. It uses a Python `src/` layout, declares package metadata in `pyproject.toml`, exposes the `promptabi` console entrypoint, ships `py.typed`, and includes focused tests for the public API, configuration loader, and CLI behavior. The implementation is intentionally small, but it is not a paper mockup: it loads real JSON configuration, normalizes artifact paths, emits deterministic typed diagnostics, returns a stable process exit code, and runs a CPU-only smoke benchmark against an example bundle.

The repository layout is designed to scale:

Path	**Purpose**
`src/promptabi`	Typed package, CLI, configuration loading, diagnostic model, renderers, and verification session.
`tests`	Focused tests for package API and CLI determinism.
`examples/minimal`	A small config, messages file, tool schema, and answer schema used by tests and docs.
`fixtures`	Seed corpus layout for tokenizer and chat-template bundles with provenance metadata.
`benchmarks`	CPU-only benchmark entrypoints for repeatable performance baselines.
`docs`	MkDocs-ready documentation for architecture, concepts, and check families.

This milestone matters because later checkers must not accrete as one-off scripts. They need stable input objects, stable diagnostics, stable renderers, stable examples, and stable tests from the beginning. PromptABI now has that surface.

3 Core Abstraction

The unifying abstraction is that LLM interface systems are compositions of finite-state transducers over byte, string, token, and structured alphabets, with partial parsers at the boundaries. A chat template maps structured messages to a string. A tokenizer maps strings to token streams and back, subject to special tokens, normalization, byte fallback, and added-token behavior. A constrained decoder or grammar describes a language of valid outputs. A stop policy cuts a prefix from that language. An application parser maps the resulting bytes back to an abstract syntax tree or rejects them.

Many high-value questions become language questions:

Property	**Question**
Reachability	Can a declared delimiter, stop sequence, or control token be produced under the selected tokenizer and grammar?
Over-reachability	Can the same sequence appear inside user-controlled content, a JSON string, a tool argument, or a markdown block?
Emptiness	Is the product of tokenizer assumptions and grammar constraints empty, so no valid output can be generated?
Non-forgeability	Can an attacker-controlled field render as role or provider structure after template expansion?
Must-survive	Does a required prompt segment remain after the framework truncation policy is applied?

The abstraction is not used to pretend arbitrary Python, arbitrary Jinja, and arbitrary provider behavior are finite-state. Instead, PromptABI should classify each check as sound, complete, bounded, heuristic, or abstaining. That taxonomy is crucial: it lets the tool be useful where it is precise and honest where it is not.

4 Artifact Model

PromptABI uses a typed artifact model because the same file can participate in multiple checks. A tokenizer config is needed for token-boundary analysis, special-token drift, chat-template parsing, and prompt-budget accounting. A tool definition is needed for role-boundary analysis, structured-output compatibility, provider migration checks, and budget survival. A diagnostic must therefore identify what artifact it refers to without coupling the renderer to a particular loader.

The initial public API provides `ArtifactRef`, `SourceSpan`, `WitnessTrace`, `Diagnostic`, `VerificationConfig`, `VerificationSession`, and `VerificationResult`. These types establish the compatibility contract:

- artifacts have stable kinds, names, optional paths, and optional versions;
- spans are one-based and validated at construction time;
- witnesses are explicit traces, not prose hidden in the message string;
- diagnostics have rule IDs, severities, artifacts, spans, witnesses, and suggestions;
- results serialize to deterministic dictionaries for JSON and snapshots.

The minimal config loader already exercises the model by resolving local artifact paths relative to the config file. This small behavior prevents a future class of flaky tests and confusing CLI output: diagnostics refer to the same canonical artifact no matter where the command is launched from.

5 Diagnostic Contract

PromptABI's diagnostic contract is designed for CI and later SARIF integration. A useful finding is not merely a line of text. It needs a stable rule ID, a severity, a precise artifact reference, a source span when available, a witness that reproduces the failure, and a suggested remediation. If two releases emit different diagnostics for the same artifact bundle, that difference should be intentional, reviewable, and testable.

The repository currently renders diagnostics in text and JSON. The JSON path is important because it fixes ordering, key names, and optional-field behavior before high-volume checkers are introduced. A future role-forgery finding can reuse the same shape:

```
{
  "rule_id": "role-boundary-nonforgeability",
  "severity": "error",
  "message": "user.content can render an assistant delimiter",
  "artifact": {"kind": "chat-template", "name": "tokenizer_config"},
  "witness": {
    "summary": "malicious user content reaches assistant header",
    "steps": ["render template", "tokenize prompt", "locate forged boundary"]
  }
}
```

The first implemented rule, `repository-skeleton`, is intentionally informational. It proves the machinery works without claiming a security or compatibility property that has not yet been implemented. That restraint is important for trust: checkers should only claim properties that the code has actually proved.

6 CLI and Embedding API

The command line entrypoint is:

```
promptabi verify --config examples/minimal/promptabi.json
```

In the current milestone the workflow validates repository wiring and artifact existence. It exits with code 0 when no error-level diagnostics are present, code 1 when verification finds error-level diagnostics, and code 2 when the config is invalid. This convention leaves room for CI to distinguish malformed invocation from a real contract failure.

The same behavior is available through the embedding API:

```
from promptabi import VerificationSession

result = VerificationSession.from_config_file("promptabi.json").run()
if not result.ok:
    raise SystemExit(1)
```

The point is not that the current checker is deep. The point is that all later checkers have an integration path. A LangChain plugin, a pre-commit hook, a GitHub Action, a notebook visualization, and a provider-fixture replay harness can all depend on the same session and result objects. That prevents the common research-tool failure mode where each experiment has its own incompatible input format and output convention.

7 Role-Boundary Non-Forgeability

The first major formal checker will analyze whether user, tool, RAG, or other attacker-controlled fields can render as structural prompt boundaries. In ChatML-like templates the relevant strings may be `<|im_start|>`, `<|im_end|>`, role names, or assistant prefixes. In Llama-style templates they may be header IDs and end-of-turn sentinels. In instruction-tag templates they may be `[INST]`, `[/INST]`, or custom fine-tune delimiters. In XML-like tool formats they may be tool-call start and end tags.

The checker should model the rendered prompt as regions: system, user, assistant, tool, developer, provider envelope, and generated assistant prefix. Then it should ask whether content controlled by one region can create syntax that is interpreted as another region. A witness should include a minimized malicious field, a rendered excerpt, the tokenized representation, and the exact boundary that becomes ambiguous or forged.

This property is structural, not semantic. PromptABI can prove that a template allows user content to create an apparent assistant turn. It cannot prove that a model will obey that turn. That distinction makes the result more useful, not less, because the finding remains valid across model weights and sampling parameters.

8 Stop and Grammar Reachability

Stop policies often look simple: if the generated text contains a string, cut there. In structured-output systems this simplicity is dangerous. A stop string can be unreachable because the tokenizer or grammar cannot produce it. It can be ambiguous because multiple token paths or decoded byte strings correspond to the same visible text. It can overreach because the string appears inside a valid JSON string, tool argument, markdown fence, or XML-like field before the structured object is complete.

PromptABI should analyze stop policies jointly with tokenizers and grammars. A useful witness for overreachability is not just "the stop appears." It is a valid output prefix, the parser state at truncation, the exact stop firing point, and the malformed or prematurely accepted structure received by the application parser. A useful witness for unreachability includes the stop sequence, its byte form, the relevant tokenizer segmentation, and the grammar state that prevents production.

This check family is attractive because it is both practical and formal. It catches flaky production bugs, and it can be stated as a reachability property over a product of tokenizer, grammar, stop automaton, and parser states.

9 Structured Output and Grammar Emptiness

Structured-output libraries increasingly compile JSON Schema, regex fragments, EBNF-like grammars, Pydantic models, or provider-specific tool schemas into a decoder constraint. The application then parses the resulting bytes with a different parser. Bugs appear when those languages do not match. A schema may be valid under a Python validator but compile into an empty grammar under a specific backend. A grammar may accept strings the application parser rejects. A tokenizer may expose Unicode, escaping, whitespace, or added-token behavior that creates ambiguous structured outputs.

PromptABI should normalize the supported JSON Schema subset into a grammar IR with source maps. It should compile that subset to automata or a backend-neutral grammar representation, then differentially compare behavior against real validators and structured-output libraries where possible. The key theorem shape is: under the supported fragment assumptions, if the product is empty, then no sequence accepted by the tokenizer and grammar backend can produce a valid application object.

Equally important is abstention. Arbitrary schemas with recursive references, custom validators, remote references, or backend-specific extensions should not be silently approximated as safe. The diagnostic should state exactly which fragment forced abstention and where that fragment appears.

10 Must-Survive Budget Verification

Token-budget failures are common because different layers count and truncate in different ways. A framework may drop old messages from the left. A RAG pipeline may trim chunks after adding metadata. A serving engine may reserve tokens for tools or generation prompts. A tokenizer mismatch may make a prompt fit in development but overflow in production. The result can be subtle: the user message survives, but the system instruction, output-format requirement, citation-bearing span, or tool schema disappears.

PromptABI should represent prompt components as named segments with must-survive constraints. A budget checker then combines segment token counts, special-token overhead, tool-definition overhead, generation reserve, and the selected framework truncation policy. The property is not "the prompt fits." The property is "all required segments survive under the policy that will be used."

A good diagnostic should show the segment table, truncation boundary, dropped fields, framework policy, tokenizer identity, and a minimized counterexample configuration. This makes the finding actionable: the developer can reduce retrieval size, reserve more budget, change the truncation policy, pin a tokenizer, or mark a different segment as required.

11 Differential Validation

PromptABI's abstractions must be checked against the libraries developers actually use. The verifier should not implement a tokenizer, chat-template engine, or grammar compiler and assume equivalence. It should differentially test its abstraction against Hugging Face tokenizers, `apply_chat_template`, `tiktoken`, SentencePiece where practical, llama.cpp metadata, vLLM and OpenAI-compatible request fixtures, and structured-output backends such as Outlines, xgrammar, llguidance, lm-format-enforcer, Guidance, and Instructor.

The testing strategy has three tiers:

- unit tests for public API and deterministic rendering;
- fixture tests for minimized artifact bundles with expected diagnostics;
- corpus tests for popular model and framework configurations pinned by revision.

Differential tests are especially valuable for version drift. A tokenizer or provider SDK update can change behavior without changing application code. PromptABI should make that drift visible as an artifact contract change rather than a mysterious downstream parsing failure.

12 Fixture Corpus

The fixture corpus is the bridge between research claims and developer trust. It should cover popular instruct-model templates, structured-output schemas, tool-call envelopes, provider fixtures, RAG budget cases, and known delimiter collision patterns. Each entry should include provenance, license notes, exact upstream revisions, hashes, minimized repro inputs, expected diagnostics, and unsupported-fragment notes.

The current repository contains the initial layout and a minimized ChatML-like tokenizer config. That seed is intentionally small, but it establishes the file shape expected by future corpus entries. A mature corpus will include Llama, Mistral, Qwen, Gemma, Phi, DeepSeek, Zephyr, OpenAI-style adapters, llama.cpp GGUF metadata, common fine-tune templates, real tool schemas from open-source agents, anonymized provider traces, and synthetic stress cases with known labels.

The corpus should avoid secrets and network requirements. Provider fixtures should be recorded request and response shapes, not live API calls. Model artifacts should be metadata and tokenizer files, not weights. This keeps CI fast, reproducible, and privacy-preserving.

13 Benchmark Lines

The first benchmark line is deliberately modest: repeatedly load the minimal configuration and run the skeleton verifier. Its value is not in absolute runtime; it proves that benchmarks are CPU-only, deterministic, executable from the repository, and able to fail if verification unexpectedly emits an error.

Future benchmark lines should measure:

Benchmark	**Measurement**
Tokenizer abstraction	Encode/decode metadata extraction, special-token handling, and normalization cost.
Template execution	Supported Jinja fragment rendering, symbolic execution, and abstention detection.
Automata products	Determinization, minimization, intersection, emptiness, and witness extraction.
Stop analysis	Reachability and overreachability across JSON, markdown, and tool-call regions.
Budget verification	Segment accounting and truncation simulation for framework policies.
Corpus verification	Cold and warm runs across pinned artifact bundles.

Benchmarks should report runtime, memory where practical, abstention rate, diagnostic count, and witness size. The paper evaluation can then separate precision, recall, agreement, and performance rather than mixing them into a single anecdotal demo.

14 Documentation and Developer Experience

A 1000-star verification repo needs more than algorithms. It needs a first-run experience where a developer immediately understands what failed and how to fix it. The README now leads with a concrete CLI command, a CPU-only explanation, and the three high-value check families. The docs tree explains architecture, quickstart, check families, and the artifact model. The contribution guide states the central norms: deterministic output, CPU-only tests, minimized fixtures, typed public APIs, and explicit soundness boundaries.

The examples directory is equally important. PromptABI should accumulate small applications that intentionally contain bugs: role-delimiter injection, stop overreach in JSON, RAG truncation that drops citations, provider migration that changes a tool-call AST, and tokenizer drift that changes special-token behavior. Each example should include both the failing artifact bundle and the fixed version. That gives users practical recipes and gives maintainers stable regression tests.

The CLI should remain terse by default and expandable on demand. A CI run needs deterministic diagnostics and exit codes. A local debugging session needs witnesses, rendered excerpts, token tables, and fix suggestions. A future `promptabi explain` command can bridge those modes.

15 Evaluation Design

The evaluation should answer four questions.

1. Does PromptABI find real interface bugs in popular LLM application stacks?
2. Are the findings precise enough that developers would act on them?
3. Does the verifier abstain honestly when artifacts exceed the supported fragment?
4. Is the runtime low enough for pull-request CI?

The experimental setup should include labeled corpora, differential agreement tests, mutation-based stress cases, and version-drift tests. Precision and recall can be measured on synthetic and real-bug corpora where ground truth is known. Differential agreement can be measured against tokenizer libraries, chat template rendering, schema validators, grammar backends, and provider fixture replay. Runtime can be reported for individual checks and corpus-wide runs. Witness quality can be evaluated by minimization size, reproducibility, and whether the witness includes enough artifact context to file an upstream bug.

The strongest paper result would be upstreamable bugs: minimized reports that maintainers of templates, adapters, structured-output libraries, or framework integrations accept as real. That evidence is more compelling than a large synthetic benchmark alone.

16 Threat Model and Non-Goals

PromptABI's threat model is structural. Attacker-controlled content may enter user messages, tool outputs, retrieval chunks, metadata, or provider-compatible fields. The verifier asks whether that content can be represented as control structure, whether output can be truncated in a way that changes parser semantics, whether a requested structured-output language is empty or ambiguous, and whether required prompt segments survive budget policies.

The non-goals are equally important. PromptABI does not prove that a model will resist prompt injection. It does not prove that a model will choose a safe tool. It does not evaluate factuality, alignment, toxicity, or behavioral compliance. It does not require model weights or generation. If an artifact contains arbitrary code, unsupported Jinja, provider behavior not captured by fixtures, or schemas outside the supported fragment, the correct behavior is to abstain or emit a bounded finding with explicit assumptions.

This narrowness makes the tool easier to trust. A structural counterexample does not depend on random seeds, temperature, model provider, or GPU hardware. If the artifact composition admits the unsafe state, that state exists.

17 Related Systems

PromptABI is adjacent to but distinct from model execution verifiers, prompt-injection scanners, schema validators, grammar-constrained decoding libraries, and provider SDK test suites. Model execution verifiers focus on tensors, devices, dtypes, shapes, checkpoint compatibility, or export gates. Prompt-injection scanners often attempt behavioral or semantic classification. Schema validators check a schema against an instance, but not necessarily the product of schema, tokenizer, grammar backend, stop policy, and application parser. Constrained-decoding libraries enforce output languages but may not report whether their language matches the downstream parser or whether stop logic truncates inside that language.

PromptABI's niche is the composition boundary. It verifies that tokenizer, template, tool, grammar, provider, and budget artifacts agree. It should integrate with existing libraries rather than replace them: validators provide ground truth for schema fragments, tokenizer libraries provide differential oracles, and provider fixtures define offline request/response semantics.

The closest analogy is a static analyzer for ABI compatibility. The concern is not whether either side of an interface is individually well-written; it is whether their composed contract remains valid as artifacts evolve.

18 Roadmap Without Step Enumeration

The roadmap is best understood as a progression from stable surfaces to formal checks to ecosystem integration. The skeleton now provides stable surfaces. The next phase should implement artifact loaders and source-span tracking, then add the tokenizer abstraction and differential tests. After that, a finite-state automata and transducer layer can support role-boundary, stop-reachability, grammar-emptiness, and budget-survival checks. Corpus growth, lockfiles, drift detection, SARIF, GitHub Actions, pre-commit hooks, HTML reports, suppression policies, and plugin APIs turn the verifier from a library into an everyday CI tool.

The research roadmap adds reproducibility packages, executable specifications, proof sketches for supported fragments, mutation-based fuzzing, cross-version CI, and real-bug benchmark suites. The community roadmap adds contribution infrastructure, compatibility matrices, integration guides, launch assets, and governance. These activities are not separate from the technical work. They are how a formal verifier becomes durable enough for production teams to trust and for researchers to reproduce.

19 Current Smoke Results

The current codebase was validated with focused tests for the new package, configuration loader, and CLI. The CLI was also run against the minimal example configuration, producing a passing informational diagnostic. The smoke benchmark ran the skeleton verifier repeatedly on the example config and reported a deterministic JSON result. These are small but important baselines: they prove that the repository can already execute code, not just describe a future tool.

Line	**Current status**
Focused tests	Public API, config loading, and CLI behavior pass in isolation.
CLI smoke	`promptabi verify -config examples/minimal/promptabi.json` returns pass.
Benchmark smoke	Repeated config load and verification completes on CPU with JSON output.
Docs smoke	MkDocs-ready navigation and pages exist for the initial structure.

These are not the final experiments. They are the baseline benchlines that later experiments will extend: every new checker should add focused tests, fixture cases, corpus cases where relevant, and a benchmark line that can be compared across revisions.

20 Why CPU-Only Is a Strength

Many LLM quality questions are inherently behavioral and require model access. PromptABI intentionally avoids those claims. Tokenization runs on CPU. Chat-template rendering runs on CPU. JSON Schema normalization, grammar compilation, automata operations, parser checks, source-span tracking, fixture replay, and truncation simulation run on CPU. Provider compatibility can be tested from recorded fixtures. Context-window analysis depends on token counts and framework policies, not logits.

This makes PromptABI deployable in environments where GPU inference, external API calls, and prompt upload are not acceptable. A company can run the verifier inside CI on private prompt artifacts. A maintainer can run corpus checks on a laptop. A paper artifact can be reproduced without model weights. A pull request can fail because a tokenizer config changed, not because a stochastic generation happened to expose the bug once.

The CPU-only boundary also sharpens the formal claims. The tool can say "this delimiter is forgeable" or "this stop sequence can truncate valid JSON" with a structural witness. It should not say "the model is safe." That honesty is a competitive advantage.

21 Conclusion

PromptABI is positioned to verify a layer of LLM systems that is both fragile and under-tooled: the discrete interface between application data structures, prompt rendering, tokenization, constrained output, provider protocols, and application parsing. The first repository milestone establishes the engineering foundation: typed package, CLI, public API, diagnostics, tests, docs, examples, fixtures, benchmarks, and contribution guidance. The research foundation is the transducer view of prompts, tokenizers, grammars, stop policies, parsers, and budget policies.

The next milestones should deepen the artifact model and implement the first formal checkers without changing the public surface. If the project maintains deterministic outputs, minimized witnesses, clear abstention boundaries, and real corpus validation, it can become both a practical CI tool and a credible systems research contribution. The core insight is simple but powerful: many production LLM failures happen before the neural network is involved, and those failures can be found before deployment.